Javascript

- Definition: Programming language built into all browsers → no installs.
- Key: JS = like C/Python, but runs directly in browser.
- Purpose: Adds interactivity to static sites (video, 2D/3D graphics, API calls, more).
- Role: With HTML (structure) + CSS (style) → JS adds behavior.
- Uses
  - Inputs store & reuse values
  - Manipulate numbers, text, data
  - Run on user events
  - Fetch data from APIs
  - Build interactive apps
- History
  - 1995 → Brendan Eich, Netscape → *LiveScript → JavaScript* (marketing).
  - Microsoft clone = JScript in IE.
  - IE dominance (95% market share, 2000s).
  - ECMAScript stalled, revived 2008 (Google V8 engine).
  - Today: Mature, standardized across browsers.
- Problem: Static HTML/CSS → limited, non-interactive.
- Solution: Need a language all browsers run → JavaScript.
- What it adds:
  - Load new content without reloading page.
  - Live previews, interactive challenges.
  - Charts, maps, games, dynamic UI.

JavaScript

```
<!— HTML button —>
<button>I am a button</button>

<!— With JS —>
<button onclick="this.style.color='red'">I am a button</button>
```

Use

- Two ways:
  1. Inline → `<script> ... </script>` inside `<html>`.
  2. Separate file → `.js` file referenced with `<script src="file.js">`. *(most common)*
- Inline script → good for small projects, but:
  - Harder to maintain.
  - Increases HTML file size (same code reloaded each time).

JavaScript

```
JS INSIDE HTML:
<script>
// JS inside HTML
const h2 = document.querySelector("#largeBold")
h2.addEventListener("click", () => {
  console.log("I was clicked")
})
</script>
<h2 id="largeBold">I am large and bold.</h2>

SEPARATE FILES:

<!— index.html —>
```

```
<head>
  <script src="main.js"></script>
</head>
<body>
  <h2 id="largeBold">I am large and bold.</h2>
</body>

// main.js
const h2 = document.querySelector("#largeBold")
h2.addEventListener("click", () => {
  console.log("I was clicked")
})

```

Developer Tools

- F12 → Opens browser dev tools.
- Console tab → Shows errors + debug prints.
- Debug prints:
  - `console.log(value)` → normal print
  - `console.error(value)` → error print
  - `console.log("I get printed to your console tab")`

Variables

- Purpose: Store data (numbers, text, game score, position, etc.).
- Important: Variables = containers for values, not the values themselves.
- Syntax:
  - `let` → create new block-scoped variable.
  - *Identifier* = variable name.
  - `=` → assignment (left gets value of right).
- Identifiers
  - Rules
    - Use Latin chars (a-z, A-Z, 0-9, `_`).
    - Cannot start with number.
    - Cannot use reserved words (e.g., `const`, `class`).
    - Case sensitive → `firstName` ≠ `firstname`.
    - Should be descriptive, not too long.
  - Good: `id`, `lessons`, `audio2`
  - Bad:
    - `2names` → starts with number
    - `const` → reserved keyword
    - `sldfjiewofjxncv` → meaningless
    - `videooutputsignalfromtheclosetinnarnia` → too long
- Naming Convention: *lower camel case* (common in JS).
  - Start with lowercase.
  - Capitalize first letter of subsequent words.
  - Examples
    - `id`
    - `lessonName`
    - `lowerCamelCase`
- Types
  - Dynamic typing
    - No need to declare type.
    - Variables can change type at runtime (not recommended → keep 1 type per variable).

```
JS JavaScript
```

```
**Numbers** → no int/float distinction.
const universe = 42
const pi = 3.14

*Strings** → text in `" "` or `' '`.
const quote = "Look forward, not back."

**Booleans** → `true` / `false`.
const yes = true
const no = false

**Arrays** → indexed collection (0-based).
const countries = ['Norway','Sweden','Denmark']
const first = countries[0]

**Objects** → key/value pairs.
const book = {name:'Jade City', author:'Fonda Lee'}
const id = book.author
```

- Variable Declaration
  - `const` → block-scoped constant (cannot change).
  - `let` → block-scoped variable (can change).
  - `var` → function/global scoped (can change + redeclare) → *avoid in modern JS*.
- Scope
  - Defines where variables are accessible.
  - New scopes in JS: blocks `{}`, functions, modules.
  - Inner scopes → can access outer scopes.
  - Global scope → outermost, accessible anywhere in file/module.
- Block = `{ ... }`

JavaScript

```
{
  let name = "block1"
} // name only exists here

if (true) {
  console.log("this is another block2")
}

========================================

const maxRadius = 20.0 // global scope

function abc(r) {
  const pi = 3.14      // function scope
  if (r > maxRadius) {
    const result = pi * r * r  // if scope
    return result
  }
  return pi * r // OK
  return result // ReferenceError (out of scope)
}

========================================

const x = 3

function double(x) {
  const result = x * 2  // result only in this scope
  return result
```

```
}

const twenty = double(10) // 20
console.log(x)            // 3
console.log(result)       // ReferenceError (out of scope)

```

- Modern variables (`let`)
  - Use `let` for mutable variables.
  - Use `const` otherwise (default).
  - Block scoped → does not affect parent variable with same name (shadowing).

JavaScript

```
// Mutation
let counter = 100
counter = 99 // mutated

// Initialization
let value
value = 42
console.log(value) // 42

console.log(test)  // ReferenceError
let test = 99

// Scope
let env = 'prod'
{
  let country = "Norway"
  {
    let capital = "Oslo"
    console.log(country, capital) // "Norway Oslo"
  }
  country = "Sweden"              // mutation ok
  console.log(env)                // "prod"
  console.log(capital)            // ReferenceError
}

```

Constants (`const`)

- Same as `let`, but cannot reassign.
- Must be initialized immediately.
- Block scoped.
- Immutable binding → cannot change reference, but object/array contents can be mutated.

JavaScript

```
// Mutation
const age = 23
age = 24 // Error: assignment to const

// Objects/Arrays (mutable internals)
const person = { name: "Hadrian" }
person.name = "Royce"    // allowed (mutates property)
person = {}              // Error (new reference)

// Initialization
const counter = 99    // ok
const counter         // SyntaxError

```

```
// Scope (same as let)
const env = 'prod'
{
  const country = "Norway"
  {
    const capital = "Oslo"
    console.log(country, capital) // "Norway Oslo"
  }
  console.log(env)      // "prod"
  console.log(capital)  // ReferenceError
}
```

Legacy variables (`var`)
- Older variable declaration (before `let` & `const`).
- Global or function scoped (not block scoped).
- Hoisted → can use before declaration.
- Loops share the same counter.
- Avoid: use `let` or `const` instead.

JavaScript

```
// Mutation
var counter = 100
counter = 99 // mutated

// Initialization
var a
a = 99
console.log(a) // 99

// Hoisting
b = 42
console.log(b) // 42
var b          // declaration hoisted

// Scope (function vs global)
var x = 0
console.log(x) // 0

function abc() {
  var x = 2
  console.log(x) // 2
}
abc()
console.log(x)   // 0

// Blocks are NOT scopes
function test() {
  var x = 2
  console.log(x) // 2
  {
    var x = 4
    console.log(x) // 4
  }
  console.log(x)   // 4 (overwrites function x)
}
test()
```

- Functions
  - Reusable blocks of code → reduce repetition, easier to test/maintain.
  - Defined with a name, parameters (inputs), and optional return value.

- JS is dynamically typed → no need to declare param/return types.
- Functions are values → can assign to variables/constants.

```JavaScript
// Definition
function f2c(f) {
  return (f - 32) * 5 / 9
}

// Usage (invocation)
const hundred = f2c(100) // 37.7
const fifty = f2c(50)    // 10
```

- Function Syntax
  - Two main ways: arrow functions (modern) & regular functions (classic).
  - Mostly identical → but classes & methods must use regular syntax.

```JavaScript
// Arrow function
const fn = (p1, p2) => {
  return p1 + p2
}

// Arrow shorthand (1 param, 1 line)
const squared = x => x * x

// Regular function
function fn(p1, p2) {
  return p1 + p2
}

// Regular no param
function pi() {
  return 3.14
}
```

- Methods
  - Functions tied to an object/class.
  - Access internal data of the object.

```JavaScript
const reply = "I AM CALM"
console.log(reply.toLowerCase()) // "i am calm"
console.log(reply.length)        // 9
```

- Classes (JS)
  - In JS, classes = functions with `new`.

```JavaScript
function Person(name, age) {
  this.name = name
  this.age = age
  this.sayHi = to => `Hi, ${to}! My name is ${name}.`
}

const leto = new Person("Leto", 3500)
console.log(leto.sayHi("Ghanima"))
// "Hi, Ghanima! My name is Leto."
```

- Maintainability
  - Functions should do one thing well (*Linux philosophy*).
  - Small, specific functions → easier debugging, flexible, predictable.

```JavaScript
const buildRocket = () => {
  rocketScience()
  engineering()
  qualityChecking()
}
```

- Functional programming
  - Functions = first-class objects in JS.
  - Can be stored, passed, returned like other values.

```JavaScript
const loudGreet = name => "GREETINGS " + name
const silentGreet = name => "Greetings " + name

const greeter = (greetFn, name) => {
  console.log(greetFn(name))
  console.log("How are you?")
}

greeter(loudGreet, "Kalam")
// GREETINGS Kalam
// How are you?

greeter(silentGreet, "Kruppe")
// Greetings Kruppe
// How are you?
```

- Anonymous functions
  - Functions without names → used inline when passing.

```JavaScript
const greeter = (greetFn, name) => {
  console.log(greetFn(name))
  console.log("How are you?")
}

greeter(name => "GREETINGS " + name, "Kalam")
greeter(name => "Greetings " + name, "Kruppe")
```

Primitive types

- Immutable, passed by value.
- null = absence of object value.
- Types:
  - `undefined` → declared but not assigned
  - `null` → empty/intentional no-value
  - `boolean` → `true` / `false`
  - `number` → 64-bit float
  - `string` → text (immutable)
  - `bigint` → integers > safe limit of `number`
  - `symbol` → unique atom values

  Object types
- Complex structures, passed by reference.

- Common ones:
  - `Object` → key/value pairs `const obj = new Object()`
  - `Array` → list of values `const arr = new Array()`
  - `Set` → list of unique values `const set = new Set()`
  - `Date` → single moment in time `const date = new Date()`

  Operators
- Arithmetic: `+`, `-`, `*`, `/`, `%` (mod), `**` (exp)
- Assignment: `=`, `+=`, `-=`, `*=`, `/=`
- Comparison:
  - `==` loose equal (type converts)
  - `===` strict equal (no conversion)
  - `!=`, `!==` not equal
  - `<`, `>`, `<=`, `>=`
- Logical: `&&` (and), `||` (or), `!` (not)
- String: `+` concatenation
- Other: `typeof`, `instanceof`

Strings

- Represent textual data (sequence of characters).
- Primitive & immutable → methods return new strings.
- Built-in `String` class with many methods/properties.

JavaScript

```
// Creation
const title = "Children of Dune"
const title2 = new String("Children of Dune")

// Multiline
const multi = `If you put away those who
report accurately, you'll keep only those...`

// Concatenation
const author = "Frank" + " Herbert"
const author2 = new String("Frank").concat(" Herbert")

// Capitalization
const cap = "Red Seas Under Red Skies"
console.log(cap.toLowerCase()) // red seas under red skies
console.log(cap.toUpperCase()) // RED SEAS UNDER RED SKIES

// Includes
const book = "The Lies of Locke Lamora"
console.log(book.includes("Lies")) // true

// Replace / ReplaceAll
const wrong = "The Man From Mars"
console.log(wrong.replace("Man From Mars","Martian"))
// "The Martian"

// Length
const q = `It' s easy to weep when staying far away.`
console.log(q.length) // 39

// Search (indexOf)
const chaos = `Chaos dwells like a poison...`
console.log(chaos.indexOf("poison")) // 20
console.log(chaos.indexOf("greed"))  // -1

// Substring
```

```
const quote = `I am Karsa Orlong of the Teblor.`
console.log(quote.substring(5,17)) // "Karsa Orlong"

// Split
const ingredients = "Brown sugar, 2 eggs, vanilla"
console.log(ingredients.split(','))
// ["Brown sugar"," 2 eggs"," vanilla"]

// Comparison
const a = "The heart of wisdom is tolerance."
const b = "The heart of wisdom is tolerance."
console.log(a === b) // true
console.log(a !== b) // false
console.log("a" < "b") // true

// Template literals (interpolation + coercion)
const greet = name => `Howdy, ${name}!`
console.log(greet("neighbor")) // "Howdy, neighbor!"

const title3 = "Gardens of the Moon"
const year = 1999
console.log(`${title3} - ${year}`)
// "Gardens of the Moon - 1999"
```

Numbers

- Represent whole & decimal numbers.
- Primitive & immutable → operations return new values.
- `Number` class wraps primitive type.

JavaScript

```
// Creation
const year = 2020
const year2 = new Number(2020)

// Notation
const int = 345
const float = 345.00
const scientific = 3.45e2   // 345
const hex = 0x159           // 345
const binary = 0b101011001  // 345

// Arithmetic
const add = 1 + 2       // 3
const sub = 1 - 2       // -1
const mul = 1 * 2       // 2
const div = 1 / 2       // 0.5
const mod = 9 % 4       // 1

// Comparison
const a = 123, b = 123, c = 345
console.log(a === b) // true
console.log(a === c) // false
console.log(a < c)   // true
console.log(a > c)   // false

// Rounding
const x = 12.567, y = 13.456
console.log(Math.round(x)) // 13
console.log(Math.ceil(x))  // 13
console.log(Math.floor(x)) // 12
console.log(x.toPrecision(2)) // "13"
```

```
console.log(y.toPrecision(3)) // "13.5"

// From string
console.log(parseInt("12"))    // 12
console.log(parseInt("13.4"))  // 13
console.log(parseInt("abc"))   // NaN
console.log(parseFloat("13.4"))// 13.4

```

Booleans

- Logical type: `true` or `false`
- Immutable (wrapped by `Boolean` class)
- Used in conditionals & control flow

JavaScript

```
// Creation
const a = true
const b = false

// Truthy/Falsy
// falsy values → false, 0, -0, 0n, "", null, undefined, NaN
console.log(Boolean(""))        // false
console.log(Boolean("hello"))   // true
```

Arrays

- Complex type: index-value relationship
- Zero-based indexing (`arr[0]` = first element)
- Used for lists (recommend same type for simplicity)

JavaScript

```
// Creation
const chars = ["Vin","Sazed","Elend","Lestibournes"]

// Extraction
console.log(chars[0])  // "Vin"
console.log(chars[99]) // undefined (out of bounds)

// Destructuring
const [first, second,  ... rest] = chars
console.log(first)  // "Vin"
console.log(second) // "Sazed"
console.log(rest)   // ["Elend","Lestibournes"]

// Parameter destructuring
const head = ([first]) => first
console.log(head(chars)) // "Vin"

// Insertion (mutates array)
const metals = ["Iron","Steel","Tin"]
metals.push("Pewter")                // end
metals.unshift("Zinc","Brass")       // start
console.log(metals)
// ["Zinc","Brass","Iron","Steel","Tin","Pewter"]

// Functional insertion (new arrays)
const a = [1,2,3]
console.log([ ... a,4])         // [1,2,3,4]
console.log([5,6, ... a])       // [5,6,1,2,3]
console.log(a.concat([4]))    // [1,2,3,4]
```

```
console.log([5,6].concat(a)) // [5,6,1,2,3]

// Modification
const quote = ["I","write","these","words","in","steel"]
quote[5] = "wood"   // replace
quote[10] = "stone" // expands array, gaps = undefined

// Deletion (mutates)
const horse = ["He","ate","my","horse."]
console.log(horse.pop())   // "horse."
console.log(horse.shift()) // "He"
console.log(horse)         // ["ate","my"]

// Functional deletion (non-mutate)
console.log(horse.slice(1))     // ["my"]
console.log(horse.slice(0,1))   // ["ate"]

// Iteration
const words = ["A","man","can","stumble"]
words.forEach(w => console.log(w.toUpperCase()))

// Transformation (map)
const c2f = c => (c * 9/5) + 32
const temps = [20,16,32]
console.log(temps.map(c2f)) // [68,60.8,89.6]

// map with index
console.log(["a","b","c"].map((el,i) => `${i}: ${el}`))
// ["0: a","1: b","2: c"]

// Reduction (reduce)
const arr = [1,2,3,4]
console.log(arr.reduce((acc,x) => acc+x, 0)) // 10

// Filter
const filtered = [10,20,5,30,2,40].filter(x => x >= 10)
console.log(filtered) // [10,20,30,40]

// Search
const nums = [1,2,3,4]
console.log(nums.indexOf(1)) // 0
console.log(nums.indexOf(5)) // -1
console.log(nums.find(x => x>2))  // 3
console.log(nums.find(x => x>50)) // undefined

// Sort (mutates)
const vals = [12,32,1,43]
console.log(vals.sort()) // [1,12,32,43]
console.log(vals.sort((a,b)=>b-a)) // [43,32,12,1]
```

Objects

- Complex type: string keys → values
- Values can be any type (incl. functions)
- Mutable (by reference) → properties can be added/modified/deleted

JavaScript

```
// Creation
const book = {
  title: "A Memory of Light",
  authors: ["Robert Jordan","Brandon Sanderson"]
}
```

```
// Shorthand
const title = "A Memory of Light"
const authors = ["Robert Jordan","Brandon Sanderson"]
const book2 = { title, authors }

// Extraction
console.log(book.title)  // dot notation
let key = "authors"
console.log(book[key])   // dynamic key
console.log(book.missing)// undefined

// Destructuring
const person = { firstName:"Rand", lastName:"al'Thor", color:"Pink" }
const { firstName,  ... rest } = person
console.log(firstName) // "Rand"
console.log(rest)      // {lastName:"al'Thor", color:"Pink"}

// Insertion / Modification
const book3 = {}
book3.title = "Towers of Midnight"     // dot notation
book3["authors"] = ["RJ","BS"]         // dynamic key
console.log(book3.hasOwnProperty("title")) // true

// Functional insertion (spread, non-mutate)
const original = { title:"The Dragon Reborn" }
const copy = {  ... original, year:1991 }
console.log(original.year) // undefined
console.log(copy.year)     // 1991

// Deletion
const eye = { death:"lighter than a feather", duty:"heavier than a mountain" }
console.log(eye.hasOwnProperty("death")) // true
delete eye.death
console.log(eye.death) // undefined

```

Control Flow

- Used to conditionally execute code. Prefer logical operators & truthy/falsy when possible, but sometimes imperative conditionals (if/else/switch) are the right tool.
- If → truthy condition, run block
- Else if → chain more conditions
- Else → fallback if none match
- Ternary → inline if/else as an expression
- Switch → cleaner when matching against fixed values, avoids repetition, supports fallthrough

JavaScript

```
// If
if (condition) {
  console.log("runs if truthy")
}
if (condition) singleStatement()

// Else if / Else
if (x > 10) {
  console.log("big")
} else if (x > 5) {
  console.log("medium")
} else {
  console.log("small")
```

```
}

// Ternary (expression form of if/else)
const size = x > 10 ? "big" : "small"

// Equivalent to:
let size2
if (x > 10) size2 = "big"
else size2 = "small"

// Switch
const val = 2
switch (val) {
  case 1:
    console.log("one")
    break
  case 2:
    console.log("two")
    // falls through unless break
  case 3:
    console.log("three")
    break
  default:
    console.log("other")
}
```

For Loop

- Structure: `for (init; condition; increment) { ... }`
- Runs until condition is falsy.
- All 3 parts optional → empty = infinite loop.
- Common bug: “off-by-one” → extra iteration leads to `undefined`.

JavaScript

```
for (let i = 5; i ⩾ 0; i--) console.log(i)  // 5..0

const arr = [1,2,3]
let sum = 0
for (let i = 0; i ⩽ arr.length; i++) sum += arr[i] // bug: NaN

```

While Loop

- Structure: `while (condition) { ... }`
- Iterates while condition truthy.
- Essentially a `for` loop without init or increment.

JavaScript

```
let x = 5
while (x ⩾ 0) { console.log(x); x-- }  // 5..0
```

Break / Continue

- break → exits loop completely.
- continue → skips current iteration, continues next.

JavaScript

```
for (let i = 0; i < 5; i++) {
  if (i === 2) continue
  console.log(i)  // 0,1,3,4
```

```
}

for (let i = 0; i < 5; i++) {
  if (i === 2) break
  console.log(i)  // 0,1
}

```

Asynchronous

- JS = single-threaded → one task at a time.
- Event loop handles async tasks in queue (e.g. `setTimeout`, events).

Synchronous vs Asynchronous

```javascript
// sync
console.log("start")
console.log("middle")
console.log("finished")
// output: start → middle → finished

// async
console.log("start")
setTimeout(() => console.log("middle"))
console.log("finished")
// output: start → finished → middle

```

Callbacks

- Sync callback → runs immediately (`arr.map(fn)`).
- Async callback → runs later (`addEventListener("click", fn)`).
- Problem: messy nesting → "callback hell."

Promises

- States: pending → fulfilled (`resolve`) → rejected (`reject`).
- Cleaner async flow than callbacks.

```javascript
const slowMul = (x,y) => new Promise((resolve,reject) => {
  setTimeout(() => resolve(x*y), 1000)
})

slowMul(2,3).then(r => console.log(r)) // 6 (after 1s)
console.log("Hi there")

```

Handling Promises

- `.then(fn)` → success handler.
- `.catch(fn)` → error handler.
- Chainable → each `.then` returns new Promise.

```javascript
fetch("/abc")
  .then(v => console.log("Resolved:", v))
  .catch(e => console.log("Rejected:", e))

returnsPromise()
  .then(v => step2(v))
  .then(v => step3(v))
```

```
    .catch(e => console.log("Error:", e))
```

Async/Await

- Problem: Promise chains = verbose, harder to read.
- Solution (2017): `async` + `await` → syntactic sugar on top of Promises.

async

- Marks function → always returns a Promise.
- Return value automatically wrapped in Promise.

```JavaScript
// explicit promise
const fn = () => new Promise(resolve => resolve(123))

// async sugar
const fn = async () => 123
```

await

- Only allowed inside async functions.
- Pauses until Promise settles (fulfilled or rejected).
- Makes async code look synchronous.

```JavaScript
const getPicture = async userName => {
  const id = await userApi.getId(userName)
  const server = await serverApi.getServer(id)
  const picture = await imageApi.getPicture(server, id)
  return picture
}
```

Equivalent promise chain

- Benefit: Cleaner, easier to read/write than chained `.then()`.

```JavaScript
const getPicture = userName => {
  return userApi.getId(userName)
    .then(id => serverApi.getServer(id))
    .then(server => imageApi.getPicture(server, id))
}
```

Fetching External Data

- Problem: Before 1998 → full page reload for every update (e.g. Google Docs, chat apps).
- 1998 → Microsoft adds XMLHttpRequest (XHR) → async data fetch without reload.
- 2006 → W3C standard.

  AJAX
- Asynchronous JavaScript + XML.
- Combines web techs to build async apps.

  REST (Representational State Transfer)
- 2000 → Roy Fielding defined REST architecture.
- Now standard for web APIs.

  REST Principles
- Client-server → separate UI from server logic.
- Stateless → every request self-contained.
- Cacheable → responses can be reused.

- Uniform interface → consistent rules:
  - Identification of resources.
  - Manipulation via representations.
  - Self-descriptive messages.
  - Hypermedia = engine of state.
- Layered system → components isolated from non-adjacent layers.
- Code on demand (optional) → server can send executable code.

  Resources
- Anything identifiable (text, images, objects, services).
- Can be collections or singletons.
- Identified by URIs.

  REST Methods
- GET → fetch resource.
- POST → create resource.
- PUT → replace/modify resource.
- PATCH → update part of resource.
- DELETE → remove resource.

  Data Formats
- XML → older, markup-based.
- JSON → lightweight, matches JS objects.
  - `JSON.parse(str)` → text → object.
  - `JSON.stringify(obj)` → object → text.
- Most APIs default to JSON.

Fetch API

- Built-in modern interface for fetching resources (e.g., external data).
- `fetch()` → returns a Promise → resolves to a Response (even if error like 404/500).
- Only rejects on network failure or request blocked.

  Usage

```JavaScript
const path = "https://en.wikipedia.org/w/api.php?origin=*&action=opensearch&search=Norway"

fetch(path)
  .then(response => response.json())
  .then(console.log)
```

Request options

- 2nd parameter → customize request:
  - method → GET, POST, PUT, PATCH, DELETE
  - headers → e.g. `Content-Type: application/json`
  - credentials → include cookies/credentials
  - body → stringified JSON

```JavaScript
fetch("/abc", {
  method: "POST",
  credentials: "include",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ key1: value1, key2: value2 })
})
  .then(response => response.json())
  .then(console.log)

```

Response object

- Properties:
  - `ok` → boolean (true if 200-299).
  - `status` → HTTP status code.
  - `.json()` → converts response body stream → Promise with JSON.

```javascript
fetch("/cars/23", {
  method: "PUT",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ key1: value1, key2: value2 })
})
  .then(response => response.json())
  .then(console.log)
```

Error handling

- Fetch does not reject on HTTP errors.
- Must check `response.ok`.

```javascript
const handleErrors = response => {
  if (!response.ok) throw Error(response.statusText)
  return response
}

fetch("/cars/23")
  .then(handleErrors)
  .then(response => response.json())
  .then(console.log)
```

Manipulating Documents (DOM)

- Purpose of JS → make interactive, dynamic web pages.
- For security → JS can't fully control browser → limited by Web API standards.

Browser APIs (3 main parts)

- Navigator → state + identity of browser.
- Window → browser tab/page container.
- Document → actual HTML page inside the window.

DOM (Document Object Model)

- When HTML loads → browser builds a tree structure of elements.
- Each HTML element → becomes a node (with child, parent, sibling, text nodes).
- JS can get, edit, insert, remove, or modify nodes in the DOM tree.
- Example HTML → DOM tree

```html
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>DOM example</title>
  </head>
  <body>
    <section>
      <ul>
```

```
        <li>Item 1</li>
        <li>Item 2</li>
      </ul>
      <p>This is a paragraph</p>
    </section>
  </body>
</html>

```

- This produces a DOM tree where:
  - `<html>` = root.
  - `<head>` + `<body>` = children.
  - Each tag (like `<ul>`, `<li>`, `<p>`) = node with text nodes inside.

DOM Manipulation

- JS manipulates HTML structure (DOM tree) → create, update, delete elements dynamically.
- DOM = tree of nodes (elements, text, attributes).

Selection

- querySelector → returns first match (CSS selector).
- querySelectorAll → returns all matches.

JavaScript

```
document.querySelector("p")         // first <p>
document.querySelector(".recipe")   // first class="recipe"
document.querySelector("#cake")     // first id="cake"
```

- getElementsByTagName("tag") → all elements by tag.
- getElementsByClassName("class") → all elements by class.
- getElementById("id") → single element by id.

Modification

- Change content via `.innerText`, `.innerHTML`, `.textContent`.

JavaScript

```
<h1>True peace required the presence of justice ... </h1>
<script>
  const h1 = document.querySelector("h1")
  console.log(h1.innerText.length) // 80
  h1.innerText = "Abc"
</script>
```

Creation

- createElement("tag") → new element.
- createTextNode("text") → new text node.
- Insert using `.appendChild`.

JavaScript

```
<ul>
  <li>haughty silence</li>
  <li>tears</li>
</ul>
<script>
  const list = document.querySelector("ul")
  const newLi = document.createElement("li")
  newLi.appendChild(document.createTextNode("and rage"))
```

```
  list.appendChild(newLi)
</script>
```

Insertion

- append(...) → add node(s)/text at end.
- prepend(...) → add node(s)/text at start.
- appendChild(node) → append single child node.

```JavaScript
target.append(document.createElement("p"), "Abc")
target.prepend(document.createElement("img"), "Abc")
```

Deletion

- removeChild(child) → delete child from parent.

```JavaScript
const parent = document.querySelector("#target")
const child = parent.querySelector("h2")
parent.removeChild(child)
```

- remove() → delete self directly.

```JavaScript
document.querySelector("h2").remove()
```

Styling

- Modify inline styles with `.style`.
- Modify classes with `.setAttribute` or `classList`.

```JavaScript
h2.style.color = "red"
h2.setAttribute("class", "title dark")
```

Document Events

- JS can react to user events → mouse clicks, key presses, drags, etc.
- Events = user/browser actions → we attach event listeners to respond.

Add Event Listeners

- `element.addEventListener(eventType, handlerFn)`
  - `eventType` → string (e.g. `"click"`, `"keyup"`, `"drag"`).
  - `handlerFn` → function to run when event fires.

```JavaScript
btn.addEventListener("click", () => {
  console.log("Button clicked!")
})
```

Common Event Types

- click → fired when mouse/touch button pressed & released.
- drag → fired when element/text is dragged.
- keyup → fired when a key is released.

Example: Counter Button

```javascript
<p>
  Count: <span id="count">0</span>
  <button id="plusOneButton">+1</button>
</p>

<script>
  // state
  let count = 0

  const btn = document.querySelector("#plusOneButton")
  const counter = document.querySelector("#count")

  btn.addEventListener("click", () => {
    count += 1
    counter.innerText = count
  })
</script>
```